

## Lecture 26

# Memory Decoding and Error Correction

Coincident decoding · address multiplexing · building larger memories · chip select · Hamming codes and single-error correction

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

check bits at 1, 2, 4, 8

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 7, §7.3–7.4

# Today's roadmap

1

§7.3

## Decoding

Rows and columns

2

§7.3

## Expansion

More words, wider words

3

§7.4

## Hamming code

Check bits and syndrome

4

§7.4

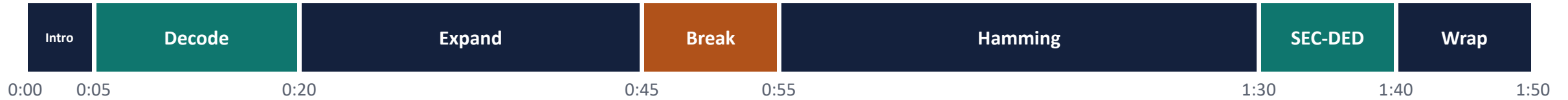
## SEC-DED

Correct one, detect two

## By the end of class you can...

- Explain coincident decoding and DRAM address multiplexing
- Build a larger memory from smaller chips
- Assign address ranges with a chip-select decoder
- Encode data with a Hamming code
- Locate and correct a single-bit error with the syndrome

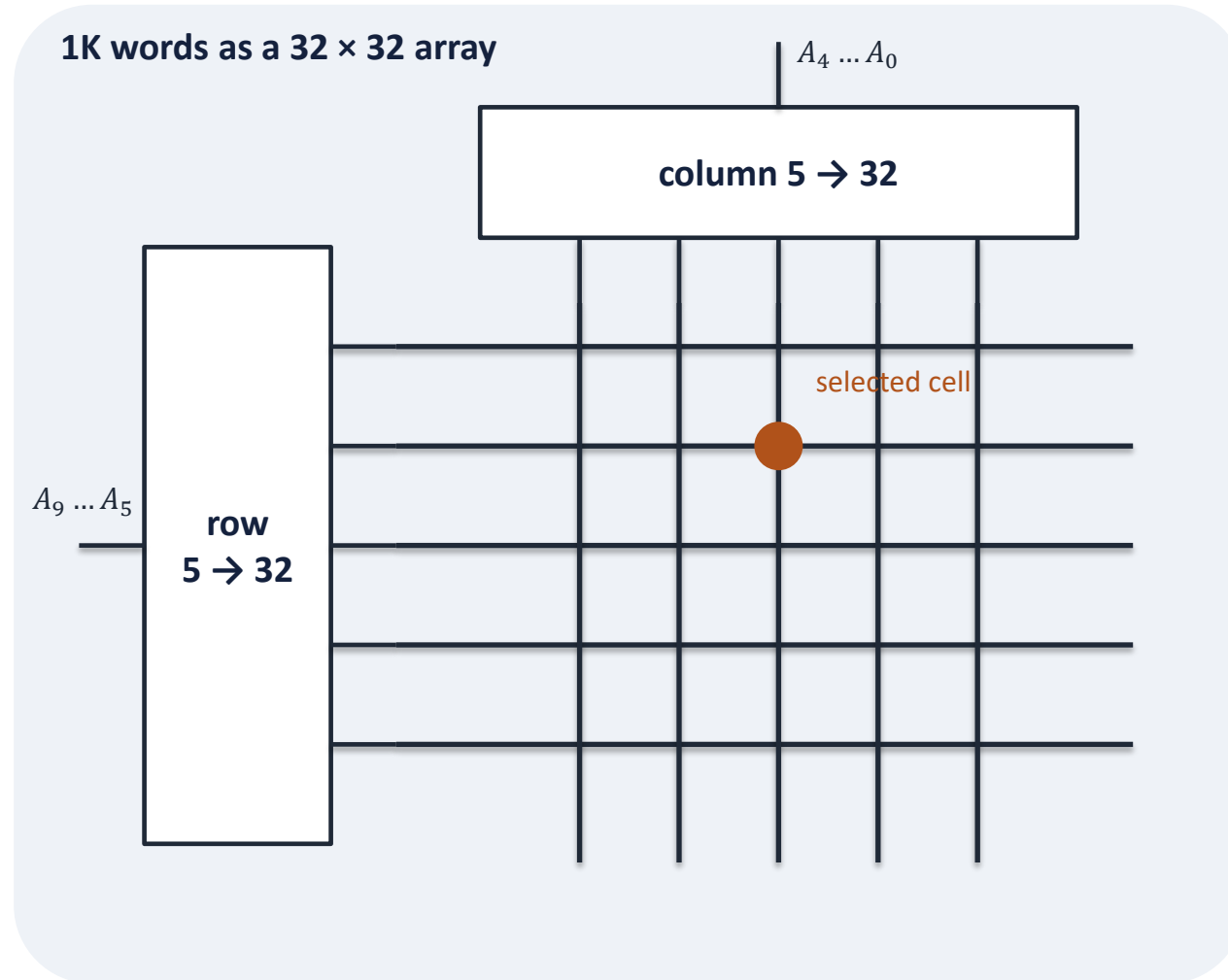
# Agenda for today's session



| Elapsed time | Topic                                     | In-class work               |
|--------------|-------------------------------------------|-----------------------------|
| 0:00 – 0:05  | Welcome, roadmap                          | —                           |
| 0:05 – 0:20  | Coincident decoding; address multiplexing | —                           |
| 0:20 – 0:45  | Larger memories; chip select              | Example 1 · Your turn 1     |
| 0:45 – 0:55  | Break                                     |                             |
| 0:55 – 1:30  | Hamming code: encode and correct          | Examples 2, 3 · Your turn 2 |
| 1:30 – 1:40  | Double-error detection                    | —                           |
| 1:40 – 1:50  | Concept check and wrap-up                 | —                           |

# Coincident decoding

§7.3



## Why two dimensions?

One 10-to-1024 decoder needs 1024 AND gates of 10 inputs.

Two 5-to-32 decoders need  $32 + 32 = 64$  gates of 5 inputs.

## DRAM address multiplexing

Send the row address, strobe RAS; then the column address, strobe CAS: half the address pins.

A  $1\text{M} \times 1$  DRAM needs 10 address pins, not 20.

# Building larger memories

§7.3

## Wider words

Put chips side by side: same address lines and chip select, separate data lines.

Two  $1\text{K} \times 4$  chips make  $1\text{K} \times 8$ .

## More words

Low address bits go to every chip; high bits drive a decoder whose outputs are the chip selects.

Four  $1\text{K} \times 8$  chips and a 2-to-4 decoder make  $4\text{K} \times 8$ .

| Target                                           | Chips ( $1\text{K} \times 8$ ) |
|--------------------------------------------------|--------------------------------|
| $1\text{K} \times 16$                            | 2 side by side                 |
| $4\text{K} \times 8$                             | 4 + 2-to-4 decoder             |
| $4\text{K} \times 16$                            | 8 + 2-to-4 decoder             |
| $16\text{K} \times 32$ from $4\text{K} \times 8$ | $4 \times 4 = 16$              |

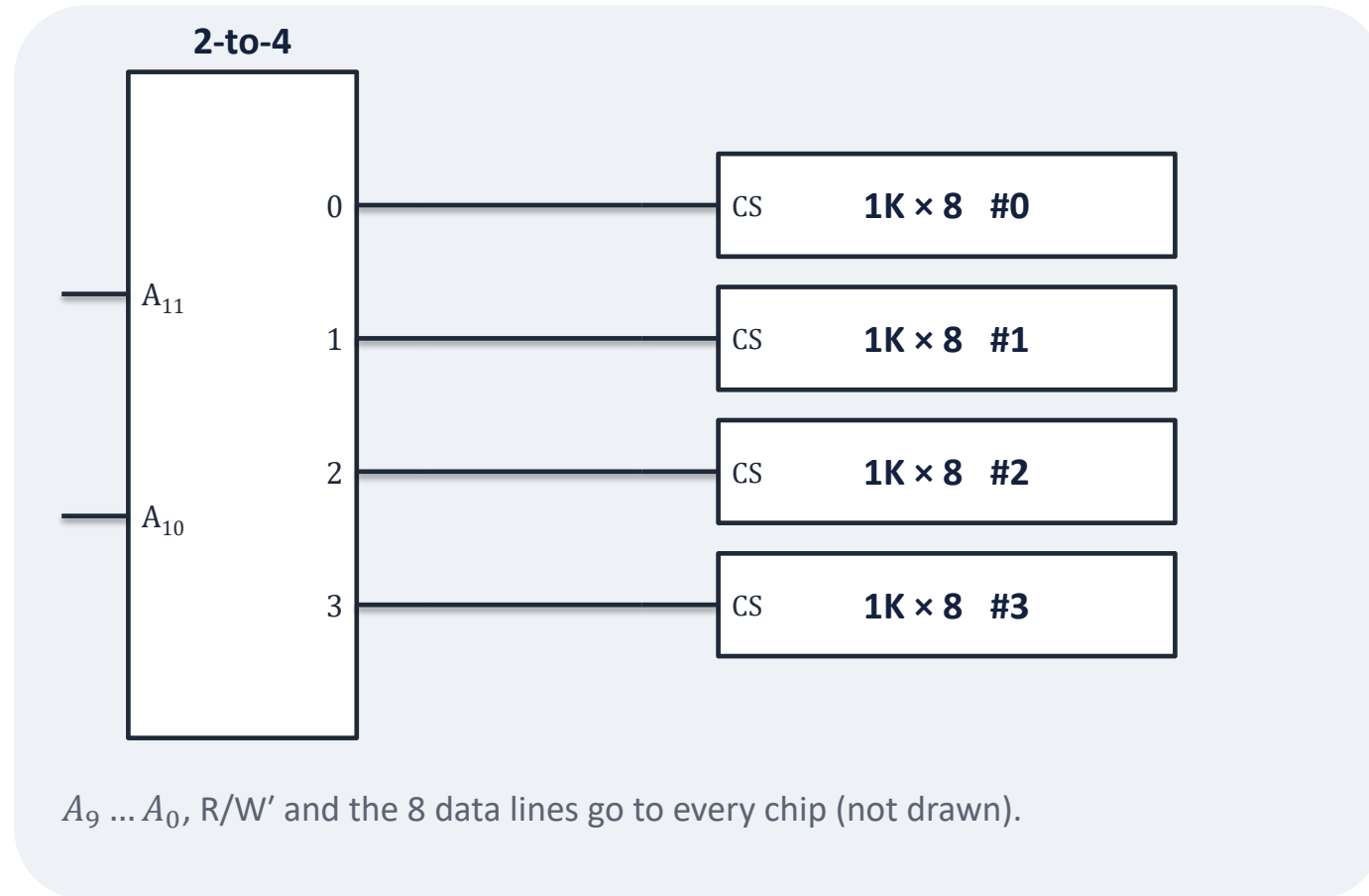
Chips needed = (target words  $\div$  chip words)  $\times$  (target width  $\div$  chip width). Outputs are three-state so the chips share the data bus.

# Example 1 — A $4K \times 8$ memory

§7.3

## EXAMPLE

Build a  $4K \times 8$  memory from  $1K \times 8$  chips. Give each chip's address range in hexadecimal.



| Chip | $A_{11}A_{10}$ | Addresses |
|------|----------------|-----------|
| #0   | 00             | 000–3FF   |
| #1   | 01             | 400–7FF   |
| #2   | 10             | 800–BFF   |
| #3   | 11             | C00–FFF   |

- 1  $4K = 2^{12}$ : 12 lines; each chip uses 10
- 2  $A_{11}A_{10}$  pick the chip through the decoder
- 3 Chip  $k$ :  $k \times 400_{16}$  to  $k \times 400_{16} + 3FF$
- 4  $9C5_{16}$ : top bits  $A_{11}A_{10} = 10 \rightarrow$  **chip #2**

# Your turn 1 — Memory expansion

§7.3

YOUR TURN

≈ 8 min

- (a) How many  $4K \times 8$  chips make a  $16K \times 32$  memory? What decoder?
- (b) A  $1M \times 1$  DRAM is a  $1024 \times 1024$  array. How many address pins with row/column multiplexing?
- (c) In Example 1, which chip holds address  $2C5_{16}$ ?

Split each address into chip-select bits and word bits.

# Your turn 1 — Solution

§7.3

## SOLUTION

- 1 (a) Words:  $16K/4K = 4$  rows of chips
- 2 Width:  $32/8 = 4$  chips side by side → **16 chips**
- 3  $16K = 2^{14}$ : 14 lines; each chip uses 12
- 4  $A_{13}A_{12}$  pick the row: a **2-to-4** decoder
- 5 (b)  $1M = 1024 \times 1024$ : 10 row bits + 10 column bits
- 6 Sent one after the other on the same pins: **10 pins**
- 7 (c)  $2C5_{16} = 0010\ 1100\ 0101$  (12 bits)
- 8  $A_{11}A_{10} = 00 \rightarrow$  **chip #0**
- 9 Low 10 bits  $10\ 1100\ 0101 = 2C5$ : word 2C5 in chip #0

# Break

10 minutes — we resume at the 0:55 mark

Next: the Hamming code

# The Hamming code

§7.4

## Idea

Add  $k$  check bits to  $n$  data bits so that the check results spell out the position of a single wrong bit.

Need  $2^k \geq n + k + 1$

## Layout

Check bits sit at positions 1, 2, 4, 8, ...; data fills the rest.

$P_{2^j}$  is the even parity of every position whose binary number has bit  $j$  set.

| Data $n$ | Check $k$ | Word $n + k$ |
|----------|-----------|--------------|
| 4        | 3         | 7            |
| 8        | 4         | 12           |
| 16       | 5         | 21           |
| 32       | 6         | 38           |
| 64       | 7         | 71           |

Overhead falls as words grow: 75 % for 4 bits, 11 % for 64 bits.

# Example 2 — Encode 8 data bits

§7.4

## EXAMPLE

Encode the data 1100 0100 with a Hamming code (even parity).

|       |       |   |       |   |   |   |       |   |    |    |    |
|-------|-------|---|-------|---|---|---|-------|---|----|----|----|
| 1     | 2     | 3 | 4     | 5 | 6 | 7 | 8     | 9 | 10 | 11 | 12 |
| 0     | 0     | 1 | 1     | 1 | 0 | 0 | 1     | 0 | 1  | 0  | 0  |
| $P_1$ | $P_2$ |   | $P_4$ |   |   |   | $P_8$ |   |    |    |    |

| Check | Positions          | Data values                             | P |
|-------|--------------------|-----------------------------------------|---|
| P1    | 1, 3, 5, 7, 9, 11  | $1 \oplus 1 \oplus 0 \oplus 0 \oplus 0$ | 0 |
| P2    | 2, 3, 6, 7, 10, 11 | $1 \oplus 0 \oplus 0 \oplus 1 \oplus 0$ | 0 |
| P4    | 4, 5, 6, 7, 12     | $1 \oplus 0 \oplus 0 \oplus 0$          | 1 |
| P8    | 8, 9, 10, 11, 12   | $0 \oplus 1 \oplus 0 \oplus 0$          | 1 |

- 1 8 data bits need 4 check bits ( $2^4 \geq 8 + 4 + 1$ ): 12 bits
- 2 Check bits at 1, 2, 4, 8; data in 3, 5, 6, 7, 9, 10, 11, 12
- 3 Each  $P$  makes the parity of its group even:
- 4  $P_1 = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$ ;  $P_2 = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$
- 5  $P_4 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$ ;  $P_8 = 0 \oplus 1 \oplus 0 \oplus 0 = 1$
- 6 Code word **0011 1001 0100**

# Example 3 — Find and fix an error

§7.4

## EXAMPLE

The word 001110010100 is received as 001100010100. Find and correct the error.

|   |   |   |   |   |   |   |   |   |    |    |    |
|---|---|---|---|---|---|---|---|---|----|----|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 1  | 0  | 0  |

| Check | Positions          | Parity | C |
|-------|--------------------|--------|---|
| C1    | 1, 3, 5, 7, 9, 11  | odd    | 1 |
| C2    | 2, 3, 6, 7, 10, 11 | even   | 0 |
| C4    | 4, 5, 6, 7, 12     | odd    | 1 |
| C8    | 8, 9, 10, 11, 12   | even   | 0 |

- 1 Recompute each check over its positions, check bit included
- 2  $C_1$ : odd  $\rightarrow 1$ ;  $C_2$ : even  $\rightarrow 0$
- 3  $C_4$ : odd  $\rightarrow 1$ ;  $C_8$ : even  $\rightarrow 0$
- 4 Syndrome  $C_8C_4C_2C_1 = \mathbf{0101} = 5$
- 5 Bit 5 is wrong: flip it 0  $\rightarrow$  1
- 6 Corrected: **001110010100** ✓ (0000 would mean no error)

# Your turn 2 — Hamming (7,4)

§7.4

YOUR TURN

≈ 10 min

- (a) Encode the 4-bit data 1010 as a 7-bit Hamming word (check bits at 1, 2, 4).
- (b) The word you sent arrives as 1011000. Find the syndrome and the corrected word.
- (c) Is 0110011 a valid code word?

Positions:  $P_1$  covers 1, 3, 5, 7;  $P_2$  2, 3, 6, 7;  $P_4$  4, 5, 6, 7.

# Your turn 2 — Solution

§7.4

## SOLUTION

1 (a) 4 data bits need 3 check bits, at 1, 2, 4

2 Data 1010  $\rightarrow$  bit 3 = 1, 5 = 0, 6 = 1, 7 = 0

3  $P_1 = b_3 \oplus b_5 \oplus b_7 = 1 \oplus 0 \oplus 0 = 1$

4  $P_2 = b_3 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 0 = 0$

5  $P_4 = b_5 \oplus b_6 \oplus b_7 = 0 \oplus 1 \oplus 0 = 1$

6 Code word **1011010**

7 (b) 1011000:  $C_1 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$

8  $C_2 = 0 \oplus 1 \oplus 0 \oplus 0 = 1$ ;  $C_4 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$

9 Syndrome  $C_4C_2C_1 = 110 = 6$ : flip bit 6  $\rightarrow$  **1011010** ✓

10 (c) 0110011:  $C_1 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$

11  $C_2 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$ ;  $C_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$

12 Syndrome **000**: a valid code word

# SEC-DED: detect two errors

§7.4

## The limit

Two errors give a nonzero syndrome that points at the wrong bit: 'correcting' it adds a third error.

## The fix

Add one overall parity bit  $P$  over the whole word: 13 bits for 8 data.

Single-error correction, double-error detection.

| Syndrome | Overall P | Conclusion              |
|----------|-----------|-------------------------|
| 0        | even      | no error                |
| $\neq 0$ | odd       | one error: correct it   |
| $\neq 0$ | even      | two errors: detect only |
| 0        | odd       | error in P itself       |

ECC memory in servers uses SEC-DED: 8 check bits protect each 64-bit word (72 bits stored).

# Concept check

1

Why do large memories use row and column decoders?

2

How are four  $1\text{K} \times 8$  chips given different address ranges?

3

How many check bits does a 16-bit Hamming code need?

4

What does a syndrome of 0 mean? Of 7?

# Key takeaways

## Coincident

row decoder × column  
decoder

## More words

high bits → chip selects

## Wider words

chips side by side

## Check bits

$$2^k \geq n + k + 1$$

## Syndrome

position of the bad bit

## SEC-DED

+1 overall parity bit

Decoders build big memories; parity keeps them honest.

Homework #11 due at the start of Lecture 28

This afternoon's lab: Lab K — NAND latch and D flip-flops

Next: ROM, PLA, PAL and programmable logic